Miyabiスーパーコンピュータのアーキテクチャ特性を極限まで引き出すBetweenness Centrality計算の超並列化および高速化手法に関する研究提案

1. 序論：大規模グラフ解析におけるBetweenness Centralityの位置づけと課題

1.1 媒介中心性（Betweenness Centrality）の学術的および実用的意義

現代の計算科学および情報科学において、大規模グラフ解析は、ソーシャルネットワークのダイナミクス解明、交通・物流ネットワークの最適化、電力網におけるカスケード障害の予測、さらには計算生物学におけるタンパク質相互作用ネットワークの分析など、多岐にわたる重要な応用領域の基盤となっている[1]。ネットワーク内における個々のノードやエッジが果たす役割の重要性を定量化する指標の中で、最も強力かつ計算コストが高いものの一つが媒介中心性（Betweenness Centrality: BC）である[3]。

グラフ理論において、媒介中心性は「ネットワーク内のすべてのノード対を結ぶ最短経路（Shortest Paths）のうち、対象となる特定のノードを経由する経路の割合」として定義される。この指標が高いノードは、異なるコミュニティ間をつなぐ「橋渡し（ブリッジ）」として機能しており、情報の拡散やシステムの脆弱性において極めて重要なボトルネック、あるいはハブであることを意味する[3]。

数学的には、無向または有向グラフ $G=(V,E)$ におけるノード v の媒介中心性 $C_B(v)$ は以下の式で表される。

$$C_B(v) = \sum_{s \neq v \neq t \in V} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

ここで、σ_{st} はノード s からノード t への最短経路の総数を表し、$\sigma_{st}(v)$ はその最短経路群のうちノード v を通過するものの数を表す。この計算をすべてのノード対に対して素朴に実行した場合、莫大な時間計算量を要する。現在、この指標を現実的な時間で計算するためのデファクトスタンダードとして用いられているのが、2001年にUlrik Brandesによって提案されたアルゴリズムである[4]。Brandesのアルゴリズムは、幅優先探索（BFS: Breadth-First Search）または単一始点最短経路（SSSP: Single-Source Shortest Path）探索のフェーズと、それに続く

依存関係の逆伝播（Back-propagation）フェーズを組み合わせることで、重みなしグラフにおいては時間計算量を $O(VE)$ 、空間計算量を $O(V+E)$ へと劇的に削減した[4]。

1.2 現行手法（単一GPUとOpenMP）の成果とアーキテクチャ上の限界

現在、修士論文に向けたアプローチとして、単一GPUとOpenMPによる並列化を組み合わせた手法を構築し、随時実行時（逐次処理）よりも高速かつ正確にBCを求めることに成功している点は、並列計算アルゴリズムの基礎を確立したという点で非常に高く評価される。OpenMPのマルチスレッド処理によるCPUコアの活用や、GPUへの処理のオフロードは、中規模のグラフデータセット（数百万ノード程度）が単一GPUのデバイスメモリ（VRAM）に収まる範囲内であれば、極めて高いスループットを発揮する。

しかしながら、この手法にはスケーラビリティとメモリ階層の観点で明確なハードウェア的限界が存在する。第一の限界は「GPUメモリの容量壁（Memory Wall）」である。現実世界の大規模ネットワーク（数十億の頂点やエッジを持つWebグラフや巨大なソーシャルグラフなど）のデータサイズは数TBに達することもあり、従来のディスクリートGPUが搭載する数十GBのVRAMには到底収まりきらない。この場合、データを分割してCPUのホストメモリからPCI Express (PCIe) バス経由で逐次転送するOut-of-Core処理が必要となるが、PCIe Gen4やGen5の帯域幅（最大で片方向64 GB/s程度）は、GPU内部のHBM（High Bandwidth Memory）の帯域幅（数TB/s）と比較して著しく低く、深刻な通信ボトルネックを引き起こす[7]。

第二の限界は「分散並列化の欠如」である。OpenMPは共有メモリ型の並列化APIであるため、単一の計算ノード内でしかスケールしない。ペタスケール、あるいはエクサスケールの計算能力を持つ現代のスーパーコンピュータの真価を引き出すには、複数ノードを跨いだ分散メモリ環境への対応（MPI等への移行）が不可欠である。

本稿では、これらの課題をブレイクスルーし、修士論文として高い新規性と国際的な学術的価値を提示するために、東京大学と筑波大学が共同運営する最先端共同HPC基盤施設（JCAHPC）の最新スーパーコンピュータ「Miyabi」のアーキテクチャ特性を極限まで引き出す、新たなBC計算アルゴリズムの再設計と実装手法を提案する[9]。

2. MiyabiシステムとGH200アーキテクチャの徹底解剖

Miyabi（雅）システムは、2025年1月に運用を開始した、倍精度理論演算性能80.1 PFLOPSを有する国内初の超並列クラスタ型スーパーコンピュータである[9]。本研究の標的とする「Miyabiの特性を生かした手法」を構築するためには、まずMiyabiシステムの特異なハードウェア構成とネットワークトポロジを深く理解する必要がある。

2.1 Miyabi-G（演算加速ノード）のハードウェア仕様

Miyabiシステムは、汎用CPUノードである「Miyabi-C」（190ノード）と、NVIDIA社製の最新ハードウェアを搭載した演算加速ノード「Miyabi-G」（1,120ノード）で構成される[9]。超高速なグラフ解析を行う上で主役となるのはMiyabi-Gである。Miyabi-Gの各ノードには、「NVIDIA

GH200 Grace Hopper Superchip」が1基搭載されている[9]。

項目	Miyabi-G (GH200搭載ノード) の主要仕様
ノード数	1,120 ノード[9]
プロセッサアーキテクチャ	NVIDIA Grace CPU (Arm Neoverse V2, 72コア) + NVIDIA Hopper H100 GPU[9]
GPUメモリ (HBM3)	96 GB (帯域幅: 4.02 TB/s)[9]
CPUメモリ (LPDDR5X)	120 GB (帯域幅: 512 GB/s) ※Miyabiシステム独自のノードあたり構成[9]
インターコネクト (ノード内)	NVLink-C2C (双方向 900 GB/s, キャッシュコヒーレント)[7]
ネットワーク (ノード間)	InfiniBand NDR200 (200 Gbps = 25 GB/s)[9]
ノード演算性能	倍精度理論演算性能: 78.8 PFLOPS (システム全体)[9]

2.2 NVLink-C2Cとハードウェア・ユニファイドメモリの革新性

GH200アーキテクチャの最大のブレイクスルーは、CPUとGPUが「NVLink-C2C (Chip-to-Chip)」と呼ばれる専用の超高速リンクで直結されている点である[12]。このリンクは双方向で900 GB/sの帯域幅を持ち、これは従来のハイエンドサーバーで用いられるPCIe Gen5の7倍以上に相当する[7]。

さらに、NVLink-C2Cはハードウェアレベルでの「キャッシュコヒーレンシ（Cache Coherency）」をサポートしている[7]。これにより、Grace CPUに接続されたLPDDR5Xメモリと、Hopper GPUに接続されたHBM3メモリが、単一の論理的および物理的なアドレス空間（ユニファイドメモリ）として統合される[14]。従来のディスクリートGPU環境で必須であった、cudaMemcpyのような明示的なデータ転送コマンドを記述することなく、GPUのストリーミングマルチプロセッサ（SM）から直接CPU側のメモリ領域へアクセス（Zero-copy access）することが可能である[20]。これは、メモリフットプリントが極めて大きくなる大規模グラフ解析において、パラダイムシフトをもたらす特性である。

2.3 InfiniBand NDRネットワークとSHARPインネットワーク・コンピューティング

Miyabiシステムのノード間通信を担うのは、最新のInfiniBand NDR200ネットワークである。各ノードはフルバイセクションバンド幅で接続されており、極めて低いレイテンシで通信が可能である[9]。

特に注目すべきは、このネットワークインフラストラクチャに実装されている「SHARP (Scalable Hierarchical Aggregation and Reduction Protocol)」プロトコルである[22]。SHARPは、分散並列計算において頻繁に発生する集団通信（Collective Operations: MPI_AllreduceやMPI_Barrierなど）の演算負荷を、ホストCPUやGPUからネットワークスイッチ内のASICへとオフロードする「インネットワーク・コンピューティング」技術である[22]。グラフ探索の各ステップにおいて、全ノード間でフロンティア（探索境界）のサイズや状態を同期する必要があるBC計算において、SHARPを活用した通信オーバーヘッドの削減は絶大な効果をもたらす。

3. 提案手法1：NVLink-C2Cとユニファイドメモリを駆使したグラフデータの階層的配置最適化

修士論文におけるコアな提案手法の第一弾として、「GH200のハードウェア・キャッシュコヒーレントなユニファイドメモリ特性を限界まで活用し、メモリウォール（容量と帯域の壁）を突破する階層的データ配置最適化」を提案する。

3.1 ページフォルトの隠蔽とデマンドページングの罠

既存の単一GPU実装では、グラフ全体がGPUのVRAMに収まることを前提としていると推測される。しかし、GH200の統合メモリ環境では、cudaMallocManagedを用いてメモリを確保するだけで、OSとハードウェアが自動的にCPU（LPDDR5X）とGPU（HBM3）間でデータを透過的にマイグレーションする[18]。Schiefferらの研究論文『A Quantitative Approach for Adopting Disaggregated Memory in HPC Systems』[19]や『Harnessing Integrated CPU-GPU System Memory for HPC』[8]によれば、GH200のシステムメモリは、従来のUnified Virtual Memory (UVM) で問題となっていたソフトウェア処理によるページフォルトのペナルティを大幅に削減できる。

しかしながら、NVIDIAの開発者フォーラムにおける実務的な検証報告[27]が示すように、開発者が単純にすべてのメモリをCPU側でアロケートし、デマンドページング（アクセスが発生した瞬間にページを移動させる仕組み）に完全に依存した場合、ページサイズの粒度（例えば8KB）でのマイグレーションがNVLink-C2C経由で頻発し、結果として従来の非ユニファイド環境（明示的コピー環境）よりも性能が低下するケースが報告されている。特にグラフ解析はメモリアクセスが極めてランダムかつ不規則（Irregular data access）であるため、このページフォルトストールが致命的なボトルネックとなる[28]。

3.2 静的・動的データの分離に基づくメモリヒント戦略

この問題を根本的に解決するため、BC計算（Brandesアルゴリズム）におけるデータ構造の特性と、GH200の異種メモリ帯域幅（HBM3: 4.02 TB/s 対 LPDDR5X: 512 GB/s）の差異を考慮し

た「静的・動的データの分離マッピング戦略」をアルゴリズムに組み込む。

BC計算で扱われるデータは、主に以下の2つのカテゴリに分類される。

1. **静的トポロジデータ（Static Topology Data）：** グラフの構造を表すデータであり、通常はCSR（Compressed Sparse Row）形式における行オフセット配列（row_offsets）と列インデックス配列（column_indices）からなる。これらは計算中に更新されることはないが、隣接ノードを探索する際に極めてランダムに読み取られ、そのデータサイズはエッジ数 E に比例して巨大になる。
2. **動的ステートデータ（Dynamic State Data）：** 探索の各ステップで頻繁に更新されるデータ。BFSツリーの深さを示す距離配列、最短経路数を保持する σ 配列、逆伝播における依存関係 δ 配列、および現在の探索キュー（フロンティア）などが含まれる。これらはノード数 V に比例するため、比較的サイズは小さいが、アトミック操作を含む激しい書き込みが発生する。

提案する具体的なメモリ配置戦略：

数千億エッジクラスの超巨大グラフを扱う際、以下のようにCUDAの高度なメモリ管理API（Memory Advise）を活用し、NVLink-C2Cのポテンシャルを最大化する。

データ構造の種別	配置先メモリ	適用するCUDA API / メモリヒント	理由および最適化の根拠
動的ステートデータ (距離, σ, δ, キュー)	GPUメモリ (HBM3, 96GB)	cudaMemPrefetchAsync を用いてGPUへ事前転送 [27]	頻繁な読み書きとアトミック操作が発生するため、4.02 TB/sの最高帯域幅を持つHBM3上に強制的に常駐させ、レイテンシを極小化する。
静的トポロジデータ (CSRエッジ配列など)	Grace CPUメモリ (LPDDR5X, 120GB)	cudaMemAdviseSetAccessedBy を用いてGPUからの直接アクセスを指定	容量が巨大でHBM3に収まらないため。NVLink-C2C (900 GB/s) の広帯域を活かし、不要なページマイグレーションを防ぎつつ、GPUのSMが直接LPDDR5X

			からキャッシュライン単位でゼロコピー・フェッチを行う[21]。

この手法を修士論文として実装・評価することで、「従来のGPUではVRAM容量の限界から計算不可能であった巨大グラフに対して、MiyabiのGH200アーキテクチャを用いることで、事前のパーティショニングなしに高速なインメモリ計算と同等の性能を達成した」という極めて強力な結論を導き出すことができる。

4. 提案手法2：Miyabi-Gにおけるマルチノード分散処理とインネットワーク・コンピューティングの統合

単一のGH200ノードで処理できる限界を超える超巨大グラフ、あるいは実用的な時間内（数分〜数秒単位）でのリアルタイム解析が求められるシナリオにおいては、Miyabi-Gの最大1,120ノードを活用したマルチノード分散並列化へのスケールアウトが必須となる[9]。ここでは、OpenMPからMPIへの拡張と、Miyabiのネットワーク特性を活かした最適化手法を提案する。

4.1 分散BFSとBrandesアルゴリズムのMPI並列化

マルチノードでのBC計算は、本質的に「分散グラフ上での多数のBFS（またはSSSP）の並行実行」に帰着する。Solomonikらの研究論文『Scaling Betweenness Centrality Using Communication-Efficient Sparse Matrix Multiplication』(SC '17)[32] で示されているように、高性能な分散BFSは、通信効率の良い疎行列-疎ベクトル積 (SpMSV) または疎行列-疎行列積 (SpGEMM) として線形代数的に定式化することが可能である。

現在のアプローチであるOpenMPから、Miyabi環境で標準提供されている「NVIDIA HPC SDK（HPC-X）」に含まれるMPIライブラリ（OpenMPIベース）を用いた分散並列モデルへの移行を行う[9]。具体的には、グラフの頂点群を複数ノードのGPU間に1次元または2次元で分割配置（1D/2D Partitioning）し、探索のフロンティアを各ステップごとにノード間で交換するアルゴリズムを構築する。

さらに、Miyabi-Gにおいて分散BCを高速化する上での鍵は、「GPU Direct RDMA」および「NVSHMEM」の活用である[34]。NVSHMEMはPGAS（Partitioned Global Address Space）プログラミングモデルを提供し、InfiniBand NDRネットワークを介してリモートノードのGPUメモリ（HBM3）に直接アクセスすることを可能にする。これにより、従来の「GPU → CPU → ネットワーク → CPU → GPU」という通信経路におけるCPUの介在を排除し、フロンティア交換の通信レイテンシを劇的に削減することができる[34]。

4.2 SHARPプロトコルを用いたAllReduce操作のハードウェアオフロード

分散BFSの同期ステップでは、「現在のステップで探索対象となるフロンティアが存在するかどうか」を全ノードで確認するためのグローバル通信（具体的には MPI_Allreduce）が毎ス

テップ発生する。ノード数が増大するにつれて、この同期オーバーヘッドがアムダールの法則（Amdahl's Law）における致命的なボトルネックとなり、並列化効率を低下させる。

これを解決するため、MiyabiシステムのInfiniBand NDRスイッチに搭載された「SHARP」プロトコルを有効化する[22]。SHARPを利用すると、各計算ノードから送信されるスカラー値（フロンティアの頂点数など）をネットワークスイッチのASIC上で直接足し合わせ（Reduction）、その集計結果だけを各ノードにブロードキャストして送り返す。 Miyabiのジョブ実行環境においてこの機能を利用するには、mpirunコマンドの実行時に以下の環境変数をジョブスクリプトで指定する必要がある[9]。

- export SHARP_SMX_ENABLED_PROTOCOLS=1
- export SHARP_SMX_PROTOCOL=1

論文『Scalable Hierarchical Aggregation and Reduction Protocol (SHARP) capabilities...』[24]の実証データが示すように、SHARPを用いることで、大規模クラスタにおけるReduceおよびAllReduceのレイテンシを最大5倍〜7倍以上削減することが可能である[24]。修士論文の評価項目として、「MiyabiにおいてSHARP機能を有効化した際と無効化した際での、BC計算のStrong Scaling（強スケーリング）曲線の比較」を含めることで、ネットワークトポロジを意識した高度なシステムチューニングの成果として高く評価されるであろう。

5. 提案手法3：GPUグラフ処理フレームワークのGH200向け拡張とMIGの活用

修士論文において、GPU用の高度なグラフ処理カーネルをゼロからC++/CUDAでフルスクラッチ記述することは、バグの温床となるだけでなく、既存の洗練された最適化（ワープレーベルのロードバランシング、アトミックフリー操作、フロンティアの動的キューイングなど）を再発明することになり非効率である。世界的なHPC研究のトレンドは、最先端のオープンソース・フレームワークを新たなハードウェアアーキテクチャに適合させるように拡張（ハック）することにある。

5.1 GunrockフレームワークのGH200向け改修

本研究のベース実装として、カリフォルニア大学デービス校（UC Davis）等によって開発されたGPU向け高性能グラフ処理ライブラリ「Gunrock」を利用することを強く推奨する[28]。Wangらによる代表的な論文『Gunrock: A High-Performance Graph Processing Library on the GPU』(PPoPP '16)[28]で発表されたこのライブラリは、バルク同期並列（BSP: Bulk-Synchronous Parallel）モデルと、頂点・エッジの「フロンティア」に対するデータ中心の抽象化（Advance, Filterなどのオペレータ）を採用しており、既にBCアルゴリズムの高効率なGPU実装が含まれている[28]。

修士論文としての独自のコントリビューションは、「標準のGunrockを単に動かすこと」ではなく、「Gunrockのメモリアロケータおよび中核カーネルを、GH200のNVLink-C2Cおよびユニファイドメモリ環境に最適化させるよう内部構造を改修すること」である。標準のGunrock

は、入力グラフのデータ（CSR構造など）をすべてGPUのデバイスメモリ（cudaMalloc）上にアロケートすることを前提としている[35]。これを改修し、前述（セクション3.2）のメモリ戦略に基づき、内部のメモリアロケータをcudaMallocManagedに置き換え、cudaMemAdvise等のヒントを注入するラッパークラスを実装する。これにより、「本来VRAM容量に束縛されるGunrockを、GH200のアーキテクチャ特性を引き出すことでテラバイト級の巨大グラフをシームレスに処理可能な『Gunrock-GH200版』へと進化させた」という明確な研究成果が生まれる。

5.2 複数MIGインスタンスを用いたマルチソース探索の並行化

さらに、Miyabi-Gの高度な利用方法として、「MIG (Multi-Instance GPU)」機能の活用が挙げられる[9]。BC計算では、正確な中心性を求めるために全ノード（またはサンプリングされた多数のノード）を始点とした最短経路探索を繰り返す必要がある。

Miyabi-Gでは、1基のH100 GPUをハードウェアレベルで論理的に分割するMIG機能を利用したキュー（regular-migなど）が用意されており、最大4つのインスタンス（例：各MIGインスタンスが18コアのCPUと約24GBのGPUメモリ、一定のSMを完全に占有）に分割できる[9]。

MIGインスタンス構成 (Miyabi-G)	**GPUリソース**	**CPUリソース**
インスタンス #0, #1, #2	32 SMs, 24 GB HBM3	18 コア, 25 GiB メモリ
インスタンス #3	26 SMs, 24 GB HBM3	18 コア, 25 GiB メモリ
出典：Miyabiシステム利用手引書 表5-4[9]		

単一の巨大なGPUコンテキストで多数のスレッドを起動して並列化するだけでなく、MIGを用いて独立した4つのGPU環境を構築し、各インスタンスに異なる始点（Source）からのBrandes探索タスクを独立して割り当てるマルチタスク戦略も極めて有効である[9]。Miyabiマニュアルに記載されているMIG対応のラッパースクリプト（set_mig_device.sh等）を利用し[9]、ノード専有実行時（regular-g）とMIG分割実行時（regular-mig）におけるスループットの比較検証を行うことは、Miyabiのシステム特性に深く根ざした優れた研究テーマとなる。

また、最新のグラフスケジューリング研究である「HD-CPS (Hardware-assisted Drift-aware Concurrent Priority Scheduler)」[37]や「OPMOS」[37]などの知見を取り入れ、複数の始点からの探索キュー（優先度付きキュー）をどのように並行処理し、無駄なワークを削減するかといったアルゴリズムレベルの改良を組み合わせることも、将来的な拡張として視野に入れるべきである[37]。

6. 修士論文執筆に向けて熟読・参照すべき必須重要論文（文献レビュー）

修士論文の「関連研究（Related Work）」および本論における理論的根拠を構築するため、以下の学術論文を精読し、本稿の文脈（Miyabi / GH200 / Graph Analytics）と関連付けて考察・引用することを強く推奨する。これらの文献を論文内で体系的にレビューすることで、研究の学術的妥当性が飛躍的に高まる。

1. GH200のメモリ管理と性能特性に関する最重要文献:

- **Schieffer, G., et al. (2024). "Harnessing Integrated CPU-GPU System Memory for HPC: a first look into Grace Hopper."** *(ICPP '24 / SC '23 関連)* [8]
  - **論文の意義:** GH200（Grace Hopper）におけるシステム統合メモリ、NVLink-C2Cのキャッシュコヒーレンシ、そしてページフォルトの挙動に関する世界初の詳細な定量評価を行った論文である。
  - **修士論文への適用:** 提案手法1（階層的データ配置）の理論的根拠として必須である。「従来のUnified MemoryとGH200のSystem Memoryの違い」を論じる際、本論文の「ページマイグレーションのオーバーヘッド」や「第一アクセス（First-touch）ポリシーの影響」に関する分析結果を引用し、自身のメモリアドバイス戦略（cudaMemAdviseの活用）の正当性を主張する。

2. GPUグラフ処理フレームワークのデファクトスタンダード:

- **Wang, Y., et al. (2016). "Gunrock: a high-performance graph processing library on the GPU."** *(PPoPP '16)* [28]
  - **論文の意義:** GPU上でグラフ処理を行うためのデータ中心型（Data-centric）アプローチと、バルク同期（BSP）モデルに基づくフレームワーク「Gunrock」のアーキテクチャを定義した記念碑的論文。
  - **修士論文への適用:** 提案手法3（GunrockのGH200向け拡張）を記述する際の基盤となる。Gunrockの「Advance（頂点の展開）」および「Filter（条件による絞り込み）」という抽象化を説明し、BC計算においてこれらのオペレータがどのように機能するかを解説する。その上で、Gunrockの本来の限界（メモリ容量制約）を、本研究がいかに打破したかを強調する。

3. マルチノード環境におけるグラフアルゴリズムの分散化:

- **Solomonik, E., et al. (2017). "Scaling Betweenness Centrality Using Communication-Efficient Sparse Matrix Multiplication."** *(SC '17)* [32]
  - **論文の意義:** スーパーコンピュータクラスの大規模分散メモリ環境において、BC計算（特に分散BFS）を疎行列演算（SpMSV）として定式化し、通信オーバーヘッドを削減する2次元パーティショニング手法を提案したトップカンファレンス論文。
  - **修士論文への適用:** 提案手法2（Miyabi-GのMPI並列化）の理論設計に用いる。MiyabiのInfiniBandネットワークを用いてノード間でフロンティアを交換する際、この論文の

知見（1D vs 2D分割の通信量モデル）を引用し、通信量 $O(N/\sqrt{P})$ の最適化について論理的な裏付けを持たせる。

4. 媒介中心性アルゴリズムの基礎:

- **Brandes, U. (2001). "A faster algorithm for betweenness centrality."** *(Journal of Mathematical Sociology)* [4]
  - **論文の意義:** 現在世界中で使われているBC計算の基礎アルゴリズム（計算量を $O(V^3)$ から $O(VE)$ に削減した手法）を確立した原典。
  - **修士論文への適用:** 序論およびアルゴリズム解説のセクションで必ず引用する。数式の定義や、前向き探索（Forward phase）と後ろ向き集計（Backward phase）の依存関係グラフ（Dependency tree）の構築メカニズムを説明する際の出典とする。

5. インネットワーク・コンピューティング（SHARP）の効果:

- **Graham, R. L., et al. (2016). "Scalable Hierarchical Aggregation Protocol (SHArP): A Hardware Architecture for Efficient Data Reduction."** [23]
  - **論文の意義:** Mellanox（現NVIDIA）のInfiniBandスイッチにおけるSHARPプロトコルのハードウェアアーキテクチャと、MPI集団通信（AllReduce等）におけるレイテンシ削減効果を実証した論文。
  - **修士論文への適用:** 提案手法2でSHARPを有効化した際のパフォーマンス向上を説明するための強力なエビデンスとなる。「グラフの探索同期において発生するAllReduceの遅延が、SHARPによってハードウェアオフロードされボトルネックが解消された」という実験結果を考察する際の拠り所とする。

6. AI/HPC融合アーキテクチャにおける最適化の最新動向 (2024-2025):

- **Xuan, W., et al. (2024/2025). "SuperOffload: Unleashing the Power of Large-Scale LLM Training on Superchips."** [16]
- **論文の意義:** GH200スーパーチップ上での大規模モデルのオフロード戦略（NVLink-C2Cを介したパラメータの投機的実行や配置最適化）に関する最新の研究。
- **修士論文への適用:** 直接グラフ処理の論文ではないが、GH200のNVLink-C2Cの挙動をモデリングし、ホストとデバイス間でどのようにデータを非同期転送・配置すべきかについての最新の最適化テクニックを学ぶことができる。本研究のメモリヒント戦略の最先端の裏付けとして応用できる。

7. 実装ロードマップと性能評価（プロファイリング）計画

修士論文を完成させるためには、単にコードを書いて速くなったと報告するだけでなく、「なぜMiyabiの特性によって速くなったのか」を科学的かつ定量的に証明するプロセスが必要である。以下のステップでの実装および評価を計画する。

7.1 実装フェーズの段階的移行

1. **フェーズ1：Miyabi環境へのベースライン移植とプロファイリング（1ヶ月）**
   - 現在所有している単一GPU+OpenMPのコードを、Miyabi-Gのインタラクティブジョブ（interact-g キュー）環境に移植する[9]。
   - コンパイラ環境として、Miyabi推奨の「NVIDIA HPC SDK（module load nvidia、module load nv-hpcx）」を利用し、強力な最適化オプション（-fast や -O3）を適用した際のベースライン性能を測定する[9]。
   - この時点で、GPUのVRAMに収まらないサイズのグラフデータを与え、OSの標準的なページフォルト処理によってどの程度性能が劣化するか（ベースラインの限界）をNVIDIA Nsight Systemsを用いてプロファイリングする[39]。
2. **フェーズ2：ユニファイドメモリ最適化とGunrockの改修（2〜3ヶ月）**
   - Gunrockのリポジトリをフォークし、内部のアロケータをcudaMallocManagedに置き換える。
   - CSRの静的配列に対して cudaMemAdviseSetAccessedBy を、動的配列に cudaMemPrefetchAsync を適用するコードを実装する[27]。
   - GAP Benchmark Suite（Twitter, Friendster, Webデータなど）を用いて、HBM3（96GB）に収まるグラフと収まらない超巨大グラフでの実行時間（TEPS: Traversed Edges Per Second）を測定・比較する[40]。
3. **フェーズ3：分散MPI拡張とSHARPインテグレーション（2ヶ月）**
   - バッチジョブ（regular-g キュー）を利用し、Miyabi-GでのMPI並列化コードを実装する[9]。ノード数を 1, 2, 4, 8, 16, 64 と増減させ、Strong Scaling（強スケーリング：問題サイズ固定での速度向上）および Weak Scaling（弱スケーリング：ノードあたりの問題サイズ固定での性能維持）を評価する。
   - ジョブ投入オプション（export SHARP_SMX_PROTOCOL=1 等）を切り替え、SHARP有効/無効時の通信オーバーヘッドの差異を計測する[9]。

7.2 評価指標とプロファイリング技術の活用

論文の説得力を高めるため、実行時間以外の以下の多角的なメトリクスを評価軸に加える。

- **NVLink-C2Cのトラフィックとページフォルト解析：**
  NVIDIA Nsight SystemsおよびNsight Computeを活用し、Grace CPUからHopper GPUへのページフォルト発生回数や、NVLink-C2C経由の帯域幅利用率（GB/s）を時系列で可視化する。これにより、提案したメモリ配置ヒント（Memory Advise）がOSレベルで意図通りに機能し、LPDDR5Xからの直接フェッチがゼロコピーで高効率に行われていることを証明する。
- **通信オーバーヘッドの定量化（MPI Profiling）：** Miyabi環境で提供されるArm MAPやIntel Trace Analyzer & Collector等のプロファイラを用い、ノード間通信におけるMPI_Allreduceの実行時間の割合を測定する[9]。SHARP導入によるインネットワーク集計の恩恵をチャート化して示す。
- **電力・エネルギー効率の評価（Green HPCの視点）：** GH200アーキテクチャは、従来のx86 CPU + ディスクリートGPUの構成に比べて、電力効率が最大数倍高いことが知られている[39]。Miyabiのバッチジョブオプションにおける mail_power_info=true などの機能で得られる電力消費情報を利用し、「消費電力1ワットあたりの計算性能（GTEPS/Watt）」と

いう観点からの評価を加える[9]。これにより、昨今の脱炭素社会に向けたグリーンHPCに貢献する研究としての付加価値を修士論文に持たせることができる[44]。

8. 結論

本研究提案では、Miyabiスーパーコンピュータ（JCAHPC）の演算加速ノード「Miyabi-G」が備える最先端のアーキテクチャ特性を最大限に引き出し、Betweenness Centrality計算を飛躍的に高速化・大規模化するための包括的な手法を論じた。

現在の単一GPUおよびOpenMPを用いたベースライン実装から脱却し、GH200 Superchipの最大の強みである「NVLink-C2CによるCPU-GPU間の超広帯域接続」と「ハードウェア支援型ユニファイドメモリ」を駆使することで、従来のGPUメモリ容量（VRAMの壁）に縛られることなく、テラバイト級の巨大グラフに対するインメモリ同等の高速処理が可能となる。静的データと動的データを分離する階層的メモリ配置戦略は、ページフォルトによる深刻なパフォーマンス低下を回避するための最適解となる。

さらに、MiyabiのInfiniBand NDRネットワークおよびSHARPプロトコルを統合したMPI分散並列化により、ノード間のフロンティア同期に伴う通信オーバーヘッドをネットワークハードウェア側へオフロードし、数千スケールのマルチノード分散処理においても高いスケーラビリティを維持することができる。

本稿で提示したロードマップに沿って、Gunrock等の既存の高度なグラフ処理フレームワークをGH200向けに改修・拡張し、Nsight Systems等を用いた徹底的なプロファイリングを通じてメモリ・ネットワークの挙動を解明していくことを推奨する。これらのアプローチを統合した研究成果は、並列分散計算およびグラフアルゴリズムの分野において極めて独自性が高く、国際的なハイパフォーマンスコンピューティングの学術会議（SC, PPoPP, IPDPS等）でも高く評価される水準の修士論文を構築する確固たる基盤となるはずである。

引用文献

1. Scalable and High Performance Betweenness Centrality on the GPU - David A. Bader, 2月 28, 2026にアクセス、https://davidbader.net/publication/2014-m-ba/2014-m-ba.pdf
2. Intrinsic Correlation with Betweenness Centrality and Distribution of Shortest Paths - MDPI, 2月 28, 2026にアクセス、https://www.mdpi.com/2227-7390/10/14/2521
3. Full article: How generative artificial intelligence empowers research in the energy economy field: a scientific knowledge map analysis based on CiteSpace - Taylor & Francis, 2月 28, 2026にアクセス、https://www.tandfonline.com/doi/full/10.1080/17583004.2025.2595769
4. E cient and Rigorous Algorithms for the Analysis of Large Temporal Networks, 2月 28, 2026にアクセス、https://www.research.unipd.it/retrieve/3bfcde10-bae1-41fa-923b-f6851a482dff/mainA.pdf

5. (PDF) A Fast Algorithm for Streaming Betweenness Centrality - ResearchGate, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/261394377_A_Fast_Algorithm_for_Streaming_Betweenness_Centrality
6. Centrality Learning: Auralization and Route Fitting - MDPI, 2月 28, 2026にアクセス、 https://www.mdpi.com/1099-4300/25/8/1115
7. NVIDIA Grace Hopper Superchip Architecture In-Depth | NVIDIA Technical Blog, 2月 28, 2026にアクセス、 https://developer.nvidia.com/blog/nvidia-grace-hopper-superchip-architecture-in-depth/
8. Harnessing Integrated CPU-GPU System Memory for HPC: a first look into Grace Hopper, 2月 28, 2026にアクセス、 https://arxiv.org/html/2407.07850v1
9. Miyabi.pdf
10. Center for Computational Sciences, University of Tsukuba, 2月 28, 2026にアクセス、 https://www.ccs.tsukuba.ac.jp/wp-content/uploads/sites/14/2025/06/585c5d8eecd18a581978d442ad4bf0ce.pdf
11. High-Dimensional Tensor Renormalization Group with Tensor Train Lattice | Progress of Theoretical and Experimental Physics | Oxford Academic, 2月 28, 2026にアクセス、 https://academic.oup.com/ptep/advance-article/doi/10.1093/ptep/ptaf193/8405689
12. NVIDIA GH200 GPU Guide: Use Cases, Architecture & Buying Tips - Clarifai, 2月 28, 2026にアクセス、 https://www.clarifai.com/blog/nvidia-gh200-gpu-guide/
13. (PDF) Understanding Data Movement in Tightly Coupled Heterogeneous Systems: A Case Study with the Grace Hopper Superchip - ResearchGate, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/383279936_Understanding_Data_Movement_in_Tightly_Coupled_Heterogeneous_Systems_A_Case_Study_with_the_Grace_Hopper_Superchip
14. NVIDIA GH200 Explained: Specs, Pricing, and Cloud Availability (2025) - Fluence Network, 2月 28, 2026にアクセス、 https://www.fluence.network/blog/nvidia-gh200/
15. BUILDING JUPITER: FROM PROCUREMENT TO DEPLOYMENT, 2月 28, 2026にアクセス、 https://www.r-ccs.riken.jp/wp/wp-content/uploads/2025/12/FugakuNEXT_11th_ApplicationSeminar.pdf
16. SuperOffload: Unleashing the Power of Large-Scale LLM Training on Superchips - arXiv, 2月 28, 2026にアクセス、 https://arxiv.org/html/2509.21271v1
17. Towards Memory Disaggregation via NVLink C2C: Benchmarking CPU-Requested GPU Memory Access, 2月 28, 2026にアクセス、 https://hpi.de/oldsite/fileadmin/user_upload/fachgebiete/rabl/publications/2025/hcds25-Werner-GraceHopper.pdf
18. Unified Memory Mechanism - Emergent Mind, 2月 28, 2026にアクセス、 https://www.emergentmind.com/topics/unified-memory-mechanism

19. Evaluating Emerging CXL-enabled Memory Pooling for HPC Systems | Request PDF, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/367400802_Evaluating_Emerging_CXL-enabled_Memory_Pooling_for_HPC_Systems
20. Nvidia Grace-Hopper Architecture - Argonne Training Program on Extreme-Scale Computing, 2月 28, 2026にアクセス、 https://extremecomputingtraining.anl.gov/wp-content/uploads/sites/96/2023/08/ATPESC-2023-Track-1-Talk-4-Giri-Nvidia-Grace-Hopper-1.pdf
21. CUDA Programming Guide - NVIDIA Documentation, 2月 28, 2026にアクセス、 https://docs.nvidia.com/cuda/cuda-programming-guide/pdf/cuda-programming-guide.pdf
22. Nezha: Breaking Multi-Rail Network Barriers for Distributed DNN Training - arXiv, 2月 28, 2026にアクセス、 https://arxiv.org/html/2405.17870v2
23. Scalable Hierarchical Aggregation Protocol (SHArP): A Hardware Architecture for Efficient Data Reduction | Request PDF - ResearchGate, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/312964483_Scalable_Hierarchical_Aggregation_Protocol_SHArP_A_Hardware_Architecture_for_Efficient_Data_Reduction
24. Scalable MPI Collectives using SHARP: Large Scale Performance Evaluation on the TACC Frontera System, 2月 28, 2026にアクセス、 https://par.nsf.gov/servlets/purl/10249393
25. Unified GPU Memory Pool Overview - Emergent Mind, 2月 28, 2026にアクセス、 https://www.emergentmind.com/topics/unified-gpu-memory-pool
26. A Quantitative Approach for Adopting Disaggregated Memory in HPC Systems | Request PDF - ResearchGate, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/373487856_A_Quantitative_Approach_for_Adopting_Disaggregated_Memory_in_HPC_Systems
27. Performance decrease on Unified GracehopperC - NVIDIA Developer Forums, 2月 28, 2026にアクセス、 https://forums.developer.nvidia.com/t/performance-decrease-on-unified-gracehopperc/322208
28. A fast work-efficient SSSP algorithm for GPUs | Request PDF - ResearchGate, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/349473038_A_fast_work-efficient_SSSP_algorithm_for_GPUs
29. Reducing Synchronization and Communication Overheads in GPUs by Preyesh Dalmia - University of Wisconsin–Madison, 2月 28, 2026にアクセス、 https://asset.library.wisc.edu/1711.dl/6XSM6YXB5MQSH8Z/R/file-e72a2.pdf
30. Gunrock: A High-Performance Graph Processing Library on the GPU - Papers on Big Data Meets New Hardware, 2月 28, 2026にアクセス、 https://readingxtra.github.io/docs/gpu-graph/WangPPoPP2016.pdf
31. Unification of Memory on the Grace Hopper Nodes - QMUL ITS Research Blog, 2月 28, 2026にアクセス、 https://blog.hpc.qmul.ac.uk/grace-hopper-unified-memory/
32. Higher-Order Graph Databases - arXiv, 2月 28, 2026にアクセス、

https://arxiv.org/pdf/2506.19661
33. KAMI: Communication-Avoiding General Matrix Multiplication within a Single GPU, 2月 28, 2026にアクセス、 https://www.ssslab.cn/assets/papers/2025-wang-KAMI.pdf
34. The Landscape of GPU-Centric Communication - arXiv, 2月 28, 2026にアクセス、 https://arxiv.org/html/2409.09874v3
35. gunrock/gunrock: Programmable CUDA/C++ GPU Graph Analytics - GitHub, 2月 28, 2026にアクセス、 https://github.com/gunrock/gunrock
36. Characterizing Matrix Multiplication Units across General Parallel Pa erns in Scientific Computing, 2月 28, 2026にアクセス、 https://www.ssslab.cn/assets/papers/2026-lu-Cubie.pdf
37. HD-CPS: Hardware-assisted Drift-aware Concurrent Priority Scheduler for Shared Memory Multicores | Request PDF - ResearchGate, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/360665359_HD-CPS_Hardware-assisted_Drift-aware_Concurrent_Priority_Scheduler_for_Shared_Memory_Multicores
38. Dimensionality reduction in multiobjective shortest path search | Request PDF - ResearchGate, 2月 28, 2026にアクセス、 https://www.researchgate.net/publication/278049716_Dimensionality_reduction_in_multiobjective_shortest_path_search
39. Developing for NVIDIA Superchips - Stony Brook University, 2月 28, 2026にアクセス、 https://www.stonybrook.edu/commcms/ookami/_pdf/20240523_Developing_GH_SW_Public.pdf
40. Delegato: Locality-Aware Atomic Memory Operations on Chiplets - UPCommons, 2月 28, 2026にアクセス、 https://upcommons.upc.edu/bitstreams/11143a3e-c050-4d62-8e61-14d752987990/download
41. Disaggregated Memory with SmartNIC Offloading: a Case Study on Graph Processing, 2月 28, 2026にアクセス、 https://www.computer.org/csdl/proceedings-article/sbac-pad/2024/561600a159/22dkxxFBMvS
42. Boosting Mathematical Optimization Performance and Energy Efficiency on the NVIDIA Grace CPU, 2月 28, 2026にアクセス、 https://developer.nvidia.com/blog/boosting-mathematical-optimization-performance-and-energy-efficiency-on-the-nvidia-grace-cpu/
43. NVIDIA Grace CPU Delivers High Bandwidth and Efficiency for Modern Data Centers, 2月 28, 2026にアクセス、 https://developer.nvidia.com/blog/nvidia-grace-cpu-delivers-high-bandwidth-and-efficiency-for-modern-data-centers/
44. 论文清单 - Quant Wiki 中文量化百科, 2月 28, 2026にアクセス、 https://quant-wiki.com/scholar/
45. INFORMATION TECHNOLOGY CENTER, THE UNIVERSITY OF TOKYO - 東京大学情報基盤センター, 2月 28, 2026にアクセス、 https://www.itc.u-tokyo.ac.jp/pamphlet/24/240708_ITC.pdf